# Optimal Precision Formats for T4

https://gemini.google.com/app/fa4ce11c965be9ab

*User prompt: What is the best precision format for the T4 GPU on Google Colab?I have access to it on the Colab plan.*

Response: The best precision format for the NVIDIA Tesla T4 on Google Colab depends on your task, but in almost all standard deep learning workloads, **FP16 (Half Precision / Mixed Precision)** is the optimal choice.

The T4 is built on NVIDIA's **Turing architecture (Compute Capability 7.5)**, which dictates what the hardware can accelerate natively versus what it cannot.

## Workload Recommendations

| Workload | Recommended Format | Hardware Acceleration | Why It's Best |
|---|---|---|---|
| **Model Training & Fine-Tuning** | **FP16 (Mixed Precision / AMP)** | Turing 2nd Gen Tensor Cores | Delivers up to 6–8x compute throughput compared to pure FP32, halves VRAM usage for activations and weights, and avoids numerical instability when combined with loss scaling. |
| **LLM / Vision Inference (Standard)** | **FP16 (`torch.float16`)** | Turing Tensor Cores | Zero degradation in generation quality, runs fully accelerated on Turing Tensor Cores, and halves standard model weights. |
| **Inference on Large Models (VRAM Constrained)** | **INT8 / INT4 (AWQ, GPTQ, BitsAndBytes)** | Turing Tensor Cores (INT8/INT4 support) | Turing was the first architecture to introduce native INT8 and INT4 Tensor Core operations. INT8 doubles throughput over FP16, and 4-bit quantization allows fitting 7B–14B LLMs into the T4's 16 GB VRAM. |

## Formats to Strictly Avoid on T4

- **Do NOT use `bfloat16` (BF16):** Hardware-accelerated BF16 was introduced with **Ampere (Compute Capability 8.0+)**. If you specify `torch.bfloat16` on a T4, PyTorch will either throw an explicit error (`ValueError: Bfloat16 is only supported on GPUs with compute capability >= 8.0`) or fall back to slow CPU/FP32 emulation, running significantly slower than standard FP32.
- **Do NOT use FP8 / FP4:** Native 8-bit and 4-bit floating point formats require Hopper (FP8) or Blackwell (FP4) GPUs.

- **Avoid pure FP32 (`float32`):** FP32 does not leverage the T4's Tensor Cores for matrix multiplication, wasting significant compute and memory bandwidth.

## How to Use FP16 on Colab (PyTorch Example)

For training or fine-tuning, always pair FP16 with PyTorch's Automatic Mixed Precision (`torch.amp`) and a gradient scaler to prevent gradient underflow:

```
import torch

scaler = torch.amp.GradScaler('cuda')

for inputs, targets in dataloader:
    optimizer.zero_grad()

    # Cast forward pass to FP16
    with torch.amp.autocast('cuda', dtype=torch.float16):
        outputs = model(inputs)
        loss = criterion(outputs, targets)

    # Scale loss to prevent underflow in backward pass
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

For inference with Hugging Face Transformers:

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "model_name",
    torch_dtype=torch.float16,  # Use float16, never bfloat16 on T4
    device_map="auto"
)
```

**Sources**

1. Bfloat16 is only supported on GPUs with compute capability - YouTube
2. FP8 and FP4 Explained: Precision, Throughput and Cost
3. What Every User Should Know About Mixed Precision Training in

---

*User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?*

Response: The format that maximizes raw token generation throughput on the Tesla T4 is **INT4 (4-bit weight quantization)**—specifically formatted as **EXL2**, **GPTQ**, or **AWQ**, executed via specialized kernels like **ExLlamaV2** or **vLLM**.

Autoregressive token generation behaves fundamentally differently from training or prompt prefill. To understand why INT4 dominates generation speed, you have to look at the T4's hardware bottleneck.

## Why INT4 Maximizes Generation Speed

Autoregressive decoding (generating text token-by-token at batch size 1 or small batches) is almost 100% **memory-bandwidth bound**, not compute bound.

Every time the GPU generates a single token, it must fetch every single parameter of the model from VRAM into the compute cores. The theoretical generation throughput is governed by:

$$\text{Tokens/second} \approx \frac{\text{VRAM Bandwidth (bytes/sec)}}{\text{Model Size (bytes)}}$$

The Tesla T4 has a relatively modest memory bandwidth of **~300 to 320 GB/s**. By shrinking the weight representation:

| Format | Bits per Weight | 7B Model Memory Read per Token | Approximate Theoretical Throughput on T4 |
|---|---|---|---|
| **FP16** | 16 bits | ~14.0 GB | ~22 – 26 tokens/s |
| **INT8** | 8 bits | ~7.0 GB | ~45 – 50 tokens/s |
| **INT4 (AWQ / GPTQ / EXL2)** | 4 bits | ~3.5 GB | **80 – 100+ tokens/s** |

Because INT4 cuts the data moving across the bus by 75% compared to FP16, generation speed scales almost linearly. Furthermore, the T4's Turing architecture possesses dedicated **INT4 Tensor Cores** (rated at 260 TOPS), meaning it does not stall during computation.

## The Format & Engine Matchup

Not all 4-bit implementations yield fast generation. How the weights are stored and unpacked makes or breaks throughput.

### 1. EXL2 / ExLlamaV2 (Fastest Token Generation on T4)

- **Format:** EXL2 (or 4-bit GPTQ loaded with the ExLlamaV2 engine).
- **Why it's fastest:** ExLlama was written specifically to maximize low-batch decode speed on consumer and datacenter GPUs. It bypasses PyTorch overhead and uses custom CUDA kernels that stream 4-bit weights and dequantize them directly inside the Tensor Core registers.
- **Speed on T4:** Typically hits 70–100+ tokens/sec on standard 7B/8B models.

### 2. AWQ (Activation-Aware Weight Quantization) with vLLM

- **Format:** 4-bit AWQ (often labeled `-AWQ` on Hugging Face).
- **Why it's best for concurrency:** AWQ protects the top 1% of salient weight channels, maintaining better reasoning and perplexity than raw GPTQ. When paired with `vLLM`'s PagedAttention, it delivers high single-stream speed while allowing high concurrent batching on the T4.

### 3. GGUF (Q4_K_M) via llama.cpp

- **Format:** GGUF 4-bit (`Q4_K_M` or `Q4_0`).
- **Why it's useful:** Ideal if you want lightweight deployment. Compiling `llama.cpp` with CUDA on Colab delivers high tokens/sec using cuBLAS and integer GEMM kernels without deep PyTorch dependencies.

## The Common Mistake: `bitsandbytes` NF4

Avoid standard **bitsandbytes 4-bit (`load_in_4bit=True` / QLoRA NF4)** if your goal is maximum generation speed.

BitsAndBytes was designed to minimize VRAM footprint to allow training and fine-tuning on limited hardware. However, its generation kernels dequantize weights on the fly back into FP16 using generic CUDA operations. On a T4, NF4 generation is often **2x to 3x slower than native FP16**, despite using a fraction of the memory.

## Maximizing the KV Cache

At long context lengths (e.g., beyond 2,048 tokens), reading the Key-Value (KV) cache starts consuming significant memory bandwidth alongside the model weights.

- **FP16 KV Cache (Default):** Standard across most engines. Reliable, but slows down as the context approaches 4k–8k tokens.
- **INT8 KV Cache:** Engines like `vLLM` and `llama.cpp` allow quantizing the KV cache to 8-bit (`--kv-cache-dtype int8` or `--cache-type-k q8_0`). This halves KV memory consumption and bandwidth overhead during decode, preventing generation speed from cratering on longer conversations.

---